PROGRAMMING IN HASKELL

Topic 13.1 – Parsers in Haskell

What is a parser

A parser is a function that takes a String and returns one of two things:

or

`Left Error`

Parsing failed — an error message is returned instead of a value.

`Right (String, a)`

Success — remaining input string plus the parsed value.

Why return the remaining string?

Parsers can be chained: each picks up from where the previous one stopped. Running charP 'a' on "abc" gives back "bc", which a next parser can consume.

Type v newtype

type (alias)

- Just a rename — no new type created
- Fully interchangeable with original
- Cannot define typeclass instances
- No runtime difference at all
- Useful only for readability
- Example: type Error = String

vs

newtype (wrapper)

- Creates a genuinely distinct type
- Compiler treats them as separate types
- Can implement Functor, Monad, etc.
- Zero runtime cost (erased at compile)
- Enables safe abstraction & wrapping
- Example: newtype Parser a = ...

Our three types

1. Error – what we return when parsing fails

```
type Error = String
```

2. Result a – the outcome of any parser

```
type Result a  = Either Error (String, a)
```

3. Parser a – wraps a function from String to Result

```
newtype Parser a = Parser
      { runParser :: String -> Result a }
```

Our agenda

- This Slide Deck – we look at building a minimal version of a primitive parser (parser for character etc)
- Next Slide Deck -> Having seen a primitive parser, we will look at the use of a library Parser and see how this works on an XML File.

Parsing one character


```
charP :: Char -> Parser Char
charP c =
    Parser $ \input ->
      case input of
        (x:xs) | x == c  ->  Right (xs, x)
        _                ->  Left "character not found"
```

Parsing one character


```
charP :: Char -> Parser Char
charP c =
    Parser $ \input ->
      case input of
        (x:xs) | x == c  ->  Right (xs, x)
        _                ->  Left "character not found"
```

`Signature:` **charP :: Char -> Parser Char** — takes a target character c, returns a Parser Char that looks for it.

`Parser $ \input ->:` Wraps a lambda inside the Parser newtype. The lambda receives the full input string when the parser is run.

`(x:xs) pattern:` Deconstructs the input: x is the first character (head), xs is the remainder (tail). Fails if input is empty.

`| x == c  guard :` Only succeeds if x equals the target c. If not, Haskell falls through to the catch-all clause.

`Right (xs, x) :` Success — returns the remaining string xs and the matched character x, wrapped in Right.

`_  ->  Left ...:` Catch-all — if x ≠ c (or input is empty) parsing fails: Left "character not found".

Using runParser — Worked Examples

a. Find 'a' in "abc" — Success

```
runParser (charP 'a') "abc"
```

-> Right ("bc", 'a')

First char of "abc" is 'a'.
'a' == 'a' ✓ guard passes.

Returns Right ("bc", 'a'):
remaining = "bc", value = 'a'.

b. Find 'x' in "abc" — Failure

```
"runParser (charP 'x') "abc
```

-> Left "character not found"

First char of "abc" is 'a'.
'a' == 'x' ✗ guard fails.

Catch-all _ fires.
Returns Left "character not found".

runParser unwraps the Parser newtype and applies the inner function. The result is always an Either — Left for failure, Right for (remaining, value).

Alternative - Trying One Parser, Then Another

The Alternative typeclass adds two operations to a parser:

or

`empty`

A parser that always fails, regardless of input. It is the identity element for (<|>) — combining anything with empty gives back the original parser.

`(<|>)`

Run the left parser. If it fails, run the right parser on the same input. If the left succeeds, the right is never tried.

```
instance Alternative Parser where
  empty = Parser $ \_ -> Left "no parse"

  Parser p1 <|> Parser p2 = Parser $ \input ->
    case p1 input of
      Left  _      -> p2 input   -- first failed: try second on same input
      Right result -> Right result -- first succeeded: done
```

empty and <|>


```
class Applicative f => Alternative f where
empty :: f a -- failure / the identity element
(<|>) ::  f a -> f a -> f a -- "or" / choice
```

Think of it as: *"try the left, if it fails, try the right."*

```
--- Maybe – failure is Nothing
Nothing <|> Nothing = Nothing
Nothing <|> Just 2 = Just 2
Just 1 <|> Just 2 = Just 1 -- left succeeds, so use it
```

--- List — failure is [], choice is concatenation:

```
[] <|> [] = []
[] <|> [2] = [2]
[1] <|> [2] = [1, 2] -- both branches kept
```

empty and <|>


```
parseTrue <|> parseFalse
```

"try parseTrue, and if that fails, try parseFalse.

Helper function - satisfy


```
--  Parse one character satisfying a predicate.
-- This is the most general primitive; everything else is built on it.
satisfy :: (Char -> Bool) -> String -> Parser Char
satisfy p desc = Parser $ \input ->
      case input of
            (x:xs) | p x -> Right (xs, x)
            [] -> Left ("unexpected end of input, expected " ++ desc)
            (x:_) -> Left ("unexpected '" ++ [x] ++ "', expected " ++ desc)
```

Examples of use of satisfy


```
charP c = satisfy (== c) ("'" ++ [c] ++ "'") -- equivalent one-liner to previous def
```

```
-- | Parse a digit character ('0'..'9').
digit :: Parser Char
digit = satisfy isdigit ”digit”
```

```
-- | Parse a letter (a-z, A-Z).
letter :: Parser Char
letter = satisfy isAlpha "letter"
```

Run a parser


```
print $ runParser (charP 'a') "abc"
print $ runParser (charP 'x') "abc"
```

returns:

```
Right ("bc",'a')
Left "character not found"
```


ANY
QUESTIONS?